PROJECT REPORT

ANIMATED SIGNAL GENERATOR AND VISUALIZATION USING PYTHON

A live, interactive waveform generation and visualization project

Submitted by	**Suneha Rawat**
Project title	Animated Signal Generator and Visualization Using Python
Programming language	Python
Libraries	NumPy and Matplotlib

Submitted in partial fulfillment of the project / practical requirement

Submitted To: Mr. SANDEEP TIWARI

ABSTRACT

This project presents an interactive Animated Signal Generator developed in Python. It generates and visualizes eight common signal waveforms: sine, cosine, square, triangular, sawtooth, exponential decay, exponential growth, and damped sine. Users can select a signal and provide amplitude, frequency, and duration through a graphical interface. Matplotlib animates the waveform, while NumPy performs numerical calculations.

KEY POINT: The application links signal parameters with visible waveform behavior.

1. INTRODUCTION

Signals are quantities that vary with time or another independent variable. In electrical and electronics engineering, signals represent voltage, current, sound, sensor measurements, and communication data. This project provides a graphical application for generating and observing signal types. Instead of only showing a static graph, it progressively draws the waveform.

2. OBJECTIVES

- Generate mathematical waveforms using Python.
- Animate waveforms in a graphical window.
- Allow users to change amplitude, frequency, and duration.
- Understand how parameters affect signal shape.
- Demonstrate NumPy and Matplotlib for scientific computing.

3. SOFTWARE AND REQUIREMENTS

Component	Requirement
Programming language	Python 3
Numerical library	NumPy
Visualization library	Matplotlib
Editor	VS Code, IDLE, PyCharm, or similar
Execution	Desktop Python environment with GUI support

4. INPUT PARAMETERS

Parameter	Meaning	Example
Signal selection	Choose one of eight signals	Sine Wave
Amplitude (A)	Controls signal height	5
Frequency (f)	Cycles per second, measured in Hz	2
Duration	Time range displayed	2 seconds
Sampling interval (dt)	Time step between samples	0.005 seconds

SAMPLE INPUT: Signal = Sine Wave, A = 5, f = 2 Hz, duration = 2 s. Cycles = frequency × duration = 2 × 2 = 4.

5. SIGNAL TYPES AND MATHEMATICAL EQUATIONS

No.	Signal	Equation / description
1	Sine	x(t) = A sin(2πft)
2	Cosine	x(t) = A cos(2πft)
3	Square	x(t) = A sign(sin(2πft))
4	Triangular	x(t) = (2A/π) arcsin(sin(2πft))
5	Sawtooth	x(t) = A[2(frac(ft)) − 1]
6	Exponential decay	x(t) = A e^(−2t)
7	Exponential growth	x(t) = A e^(0.8t)
8	Damped sine	x(t) = A e^(−2t) sin(2πft)

6. WORKING PRINCIPLE

- Initialize amplitude, frequency, duration, and sampling interval.
- Create a time array using NumPy.
- Calculate the selected waveform from its equation.
- Create the Matplotlib plot and interactive controls.
- Animate the waveform by updating plotted data frame by frame.
- When inputs change, recalculate the signal and update the display.

7. PROJECT FLOWCHART

START → Initialize inputs → Select signal → Generate time samples and calculate x(t) → Animate/display waveform → Change inputs? → YES: Recalculate and update → NO: Close window / END

8. CODE EXPLANATION

8.1 Signal generation

The generate_signal() function receives the selected signal number, time array, amplitude, and frequency. It applies the corresponding equation and returns the waveform samples.

8.2 Animation

FuncAnimation repeatedly calls animate(). The line's plotted data is updated with samples up to the current frame, creating a progressive drawing effect.

8.3 Interactive controls

RadioButtons allow signal selection. TextBox controls accept amplitude, frequency, and duration. The update_signal() callback validates inputs and regenerates the signal.

9. COMPLETE PYTHON SOURCE CODE

Save the following source code as signal_generator.py.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from matplotlib.widgets import TextBox, RadioButtons


plt.style.use("dark_background")


# Default input values
A = 5
frequency = 2
duration = 2
signal_choice = 1
dt = 0.005
t = np.arange(0, duration, dt)


signal_names = [
    "Sine Wave", "Cosine Wave", "Square Wave", "Triangular Wave",
    "Sawtooth Wave", "Exponential Decay", "Exponential Growth",
    "Damped Sine Wave"
]


def generate_signal(choice, t, A, f):
    w = 2 * np.pi * f
    if choice == 1:
        return A * np.sin(w * t)
    elif choice == 2:
        return A * np.cos(w * t)
    elif choice == 3:
        return A * np.sign(np.sin(w * t))
    elif choice == 4:
        return (2 * A / np.pi) * np.arcsin(np.sin(w * t))
    elif choice == 5:
        return A * (2 * (f * t - np.floor(f * t)) - 1)
    elif choice == 6:
        return A * np.exp(-2 * t)
    elif choice == 7:
        return A * np.exp(0.8 * t)
    elif choice == 8:
        return A * np.exp(-2 * t) * np.sin(w * t)


# Create main animation window
fig, ax = plt.subplots(figsize=(12, 7))
plt.subplots_adjust(left=0.10, right=0.75, bottom=0.18, top=0.88)
fig.suptitle("ANIMATED SIGNAL GENERATOR", fontsize=18,
             fontweight="bold", color="cyan")


ax.set_title("Live Signal Visualization")
ax.set_xlabel("Time (seconds)")
ax.set_ylabel("Amplitude")
ax.grid(True, alpha=0.3)


y = generate_signal(signal_choice, t, A, frequency)
line, = ax.plot([], [], color="cyan", linewidth=2.5)
ax.set_xlim(0, duration)
ax.set_ylim(-A * 1.5, A * 1.5)


info_text = fig.text(
    0.77, 0.75,
    "Signal: Sine Wave\\nAmplitude: 5\\nFrequency: 2 Hz\\nDuration: 2 sec",
    fontsize=11, color="white",
    bbox=dict(facecolor="#182635", edgecolor="cyan",
              boxstyle="round,pad=0.6")
)


def animate(frame):
    line.set_data(t[:frame], y[:frame])
    return line,


ani = FuncAnimation(
```

```
    fig, animate, frames=len(t), interval=10,
    blit=True, repeat=True
)

# Signal selection controls
ax_radio = plt.axes([0.78, 0.35, 0.19, 0.35])
radio = RadioButtons(ax_radio, signal_names, active=0)

# Input boxes
ax_amp = plt.axes([0.12, 0.07, 0.15, 0.05])
amp_box = TextBox(ax_amp, "Amplitude", initial="5")
ax_freq = plt.axes([0.36, 0.07, 0.15, 0.05])
freq_box = TextBox(ax_freq, "Frequency", initial="2")
ax_dur = plt.axes([0.60, 0.07, 0.15, 0.05])
dur_box = TextBox(ax_dur, "Duration", initial="2")

def update_signal(label):
    global A, frequency, duration, t, y, signal_choice
    try:
        A = float(amp_box.text)
        frequency = float(freq_box.text)
        duration = float(dur_box.text)

        if A <= 0 or frequency <= 0 or duration <= 0:
            print("Please enter positive values.")
            return

        signal_choice = signal_names.index(label) + 1
        t = np.arange(0, duration, dt)
        y = generate_signal(signal_choice, t, A, frequency)

        ax.set_xlim(0, duration)
        maximum = max(np.max(np.abs(y)), 1)
        ax.set_ylim(-maximum * 1.2, maximum * 1.2)

        info_text.set_text(
            f"Signal: {label}\\nAmplitude: {A}\\n"
            f"Frequency: {frequency} Hz\\nDuration: {duration} sec"
        )
        ani.frame_seq = ani.new_frame_seq()
        fig.canvas.draw_idle()

    except ValueError:
        print("Invalid input. Please enter numeric values.")

radio.on_clicked(update_signal)

# Run application
plt.show()
```

10. HOW TO INSTALL AND RUN

- Install Python 3.
- Open Command Prompt or your editor terminal.
- Install dependencies: pip install numpy matplotlib
- Save the code as signal_generator.py.
- Run: python signal_generator.py
- Choose a waveform and enter input values in the controls.

11. EXPECTED OUTPUT AND OBSERVATIONS

A graphical window displays the animated waveform on the left, signal selection options on the right, and input fields along the bottom.

Change made	Expected observation
Increase amplitude	Waveform height increases.
Increase frequency	More cycles appear in the same duration.
Increase duration	The time axis becomes longer.
Exponential decay	Signal decreases toward zero.
Exponential growth	Signal rises over time.
Damped sine	Oscillation amplitude gradually decreases.

12. APPLICATIONS

- Learning signal processing and waveform properties.
- Engineering laboratory demonstrations.
- Understanding amplitude, frequency, duration, and sampling.
- Visualizing periodic and exponential signals.

13. LIMITATIONS

- Educational visualization, not a calibrated measurement instrument.
- Sharp transitions may appear slightly imperfect due to finite sampling.
- Designed for desktop Python GUI execution.
- Very large duration values may affect performance.

14. FUTURE ENHANCEMENTS

- Pause, resume, and restart controls.
- Slider controls and improved input validation.
- FFT and frequency-spectrum visualization.
- Export generated samples to CSV.
- Dashboard showing all signals simultaneously.

15. CONCLUSION

The Animated Signal Generator project was implemented in Python using NumPy and Matplotlib. It generates eight signal types and provides a graphical interface for selecting a waveform and entering signal parameters. The animation helps users understand the relationship between mathematical equations and plotted waveforms, fulfilling the objective of a visual and educational signal-generation application.

REFERENCES

- Python documentation: https://docs.python.org/3/
- NumPy documentation: https://numpy.org/doc/
- Matplotlib documentation: https://matplotlib.org/